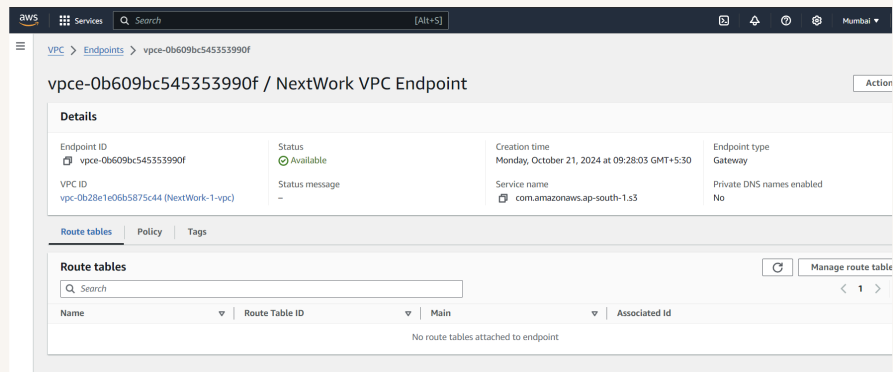




VPC Endpoints



ajv.singh3107@gmail.com





Introducing Today's Project!

What is Amazon VPC?

Amazon VPC is a service that lets you create a private, isolated network within AWS. It's useful because it provides full control over network configurations like IP ranges, subnets, and security, ensuring secure and scalable cloud infrastructure.

How I used Amazon VPC in this project

In today's project, I used Amazon VPC to create a secure network for my EC2 instance and S3 bucket. I set up a VPC endpoint to ensure traffic between them was private, preventing exposure to the public internet and improving security.

One thing I didn't expect in this project was...

One thing I didn't expect was how crucial route tables and endpoint policies are for securing traffic. Ensuring traffic routes through the VPC endpoint, not the public internet, really emphasized the importance of proper configuration.

This project took me...

This project took me 2 hours to complete. The most time-consuming part was configuring the VPC endpoint, route tables, and bucket policy to ensure secure communication.



In the first part of my project...

Step 1 - Architecture set up

In this step, we'll create a VPC, launch an EC2 instance, and set up an S3 bucket. This forms the core of your cloud setup, allowing secure communication between resources. The EC2 instance will later connect to S3 without internet access.

Step 2 - Connect to EC2 instance

We'll connect to your EC2 instance and try accessing S3 via the public internet. This helps you see how traffic flows before setting up a VPC endpoint. It's a key step to compare security and performance with vs. without an endpoint.

Step 3 - Set up access keys

To give your EC2 instance access to AWS services, you'll create access keys. These credentials will allow the instance to securely interact with AWS resources, enabling actions like using the AWS CLI or SDK to manage services.

Step 4 - Interact with S3 bucket

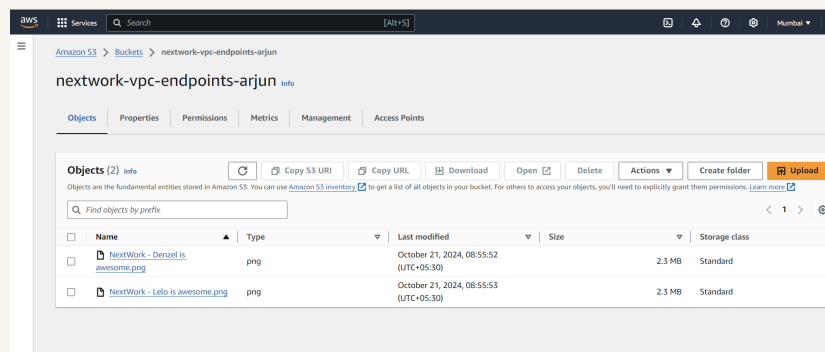
In this step, I'm going to use the access keys on the EC2 instance to securely access the S3 bucket. This demonstrates how the EC2 instance can interact with AWS services using the credentials we just set up. Let's do it!



Architecture set up

Set up a VPC (10.0.0.0/16) with one public subnet, no NAT gateway, and no VPC endpoints yet. Launch an EC2 instance with a public IP for EC2 Instance Connect. Create a security group allowing SSH.

I also set up a bucket in Amazon S3 named nextwork-vpc-endpoints-yourname. I uploaded two files: "NextWork - Denzel is awesome.png" and "NextWork - Lelo is awesome.png" into the bucket for later access via the EC2 instance.





Access keys

Credentials

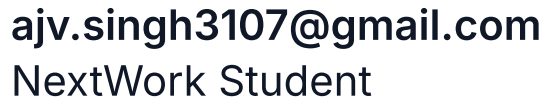
To set up my EC2 instance to interact with my AWS environment, I configured my **access key** and **default region** in the EC2 terminal. I left the **default output format** empty, so it defaults to **JSON** for displaying results.

Access keys are credentials consisting of an Access Key ID and a Secret Access Key. They allow programs or services to interact with AWS resources securely. They're essential for using the AWS CLI or SDKs to automate tasks and access services.

Secret access keys are the private part of the access credentials. They pair with the Access Key ID to authenticate and authorize API requests to AWS services. They should be kept secure, as anyone with access can perform actions on your AWS account.

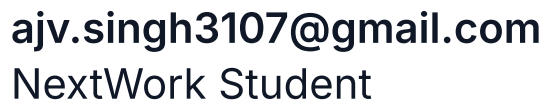
Best practice

Although I'm using access keys in this project, a best practice alternative is to use IAM roles with EC2 instances. This grants secure, temporary credentials to the instance, eliminating the need to manually manage access keys.



The terminal responded with the list of all buckets configured in S3.

[illegible]

[illegible]



Uploading objects to S3

The first command I ran was `sudo touch /tmp/nextwork.txt`. This command created an empty file named `nextwork.txt` in the `/tmp` directory of the EC2 instance.

The second command I ran was `aws s3 cp /tmp/nextwork.txt s3://nextwork-vpc-endpoints-yourname`. This command will copy the `nextwork.txt` file from the `/tmp` directory to the specified S3 bucket.

The third command I ran was `aws s3 ls s3://nextwork-vpc-endpoints-yourname`, which validated that the file `nextwork.txt` was successfully uploaded to the S3 bucket and is listed with 0 bytes.

```
aws Services Search [Alt+S] Mumbai
[ec2-user@ip-10-0-1-232 ~]$ sudo touch /tmp/nextwork.txt
[ec2-user@ip-10-0-1-232 ~]$ aws s3 cp /tmp/nextwork.txt s3://nextwork-vpc-endpoints-arjun
upload: ../../tmp/nextwork.txt to s3://nextwork-vpc-endpoints-arjun/nextwork.txt
[ec2-user@ip-10-0-1-232 ~]$
[ec2-user@ip-10-0-1-232 ~]$
[ec2-user@ip-10-0-1-232 ~]$
[ec2-user@ip-10-0-1-232 ~]$
[ec2-user@ip-10-0-1-232 ~]$
[ec2-user@ip-10-0-1-232 ~]$
[ec2-user@ip-10-0-1-232 ~]$
[ec2-user@ip-10-0-1-232 ~]$
[ec2-user@ip-10-0-1-232 ~]$
[ec2-user@ip-10-0-1-232 ~]$ aws s3 ls s3://nextwork-vpc-endpoints-arjun
2024-10-21 03:25:52      2431554 NextWork - Denzel is awesome.png
2024-10-21 03:25:53      2399812 NextWork - Lelo is awesome.png
2024-10-21 03:40:39           0 nextwork.txt
[ec2-user@ip-10-0-1-232 ~]$
[ec2-user@ip-10-0-1-232 ~]$
[ec2-user@ip-10-0-1-232 ~]$
[ec2-user@ip-10-0-1-232 ~]$
```



In the second part of my project...

Step 5 - Set up a Gateway

To create a VPC endpoint for S3, go to the VPC console, choose "Endpoints" > "Create Endpoint." Select S3 as the service, choose your VPC, configure route tables, set the endpoint policy, and then click "Create Endpoint" to enable secure communication

Step 6 - Bucket policies

In this step, we're going to create a bucket policy that restricts access to our S3 bucket, allowing only traffic from the VPC endpoint. This will confirm that the endpoint is properly configured and that no traffic is coming from the public internet

Step 7 - Update route tables

In this step, we're testing if our EC2 instance can still access the S3 bucket after limiting access to the VPC endpoint. If the EC2 instance can access the bucket, the endpoint setup is correct. If not, we'll troubleshoot the connectivity issue.

Step 8 - Validate endpoint connection

In this step, we're testing the VPC endpoint setup again to confirm the EC2 instance can access the S3 bucket securely. We'll also restrict the VPC's access to your AWS environment, ensuring all communication stays private and

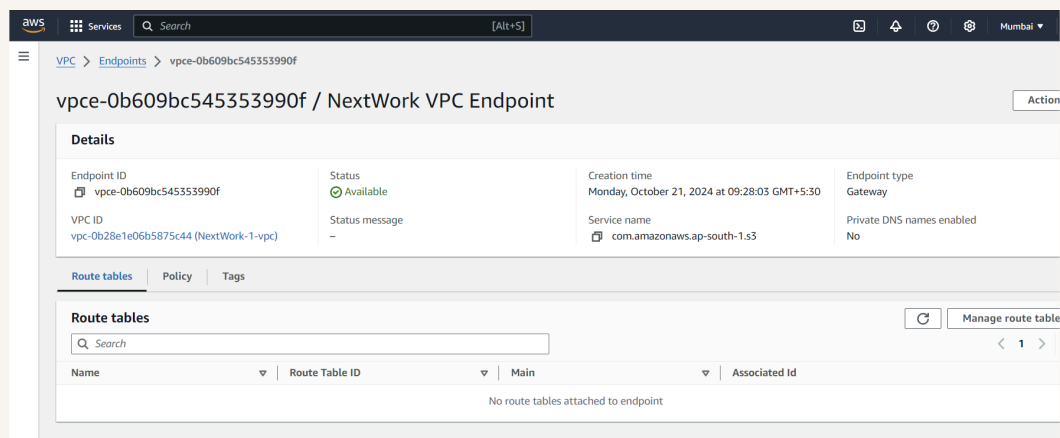


Setting up a Gateway

I set up an S3 Gateway, which is a VPC endpoint that connects your VPC to S3 over the AWS network. It allows secure, private access to S3 without going through the public internet, providing better security and reducing latency.

What are endpoints?

An endpoint is a private connection between your VPC and AWS services like S3, allowing secure communication within the VPC without using the public internet. It improves security and reduces latency by keeping traffic internal.

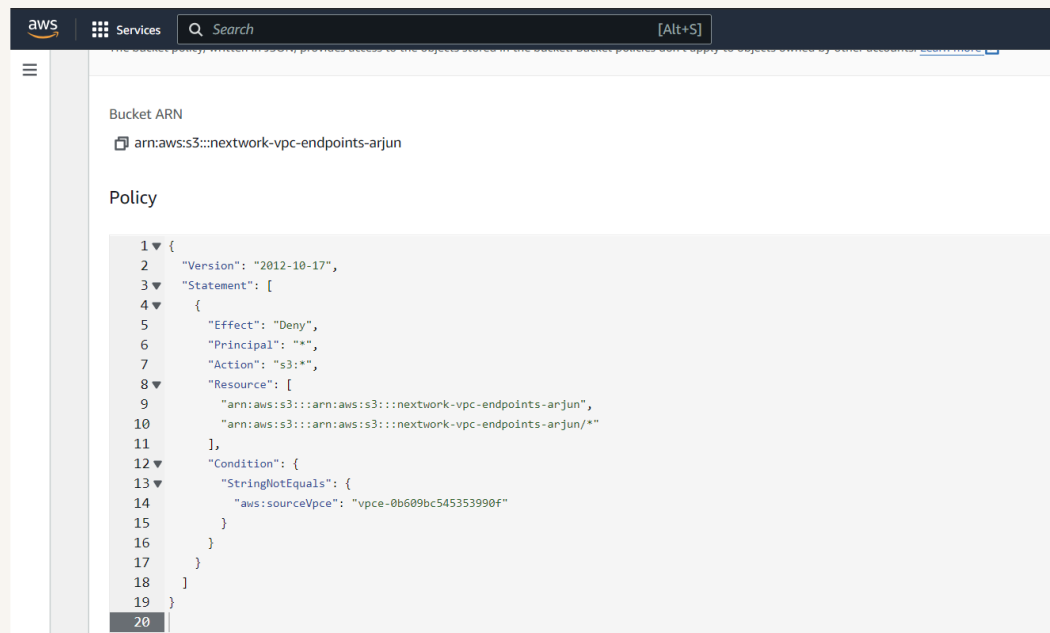




Bucket policies

A bucket policy is a set of permissions for an S3 bucket, controlling who can access it and what actions they can perform. It helps secure the bucket by defining access based on conditions like IP, VPC, or specific users.

My bucket policy will deny all access to the S3 bucket unless the request comes from a specific VPC endpoint (vpce-xxxxxxx). This ensures that only traffic from the defined VPC endpoint can access the bucket, blocking all other access.

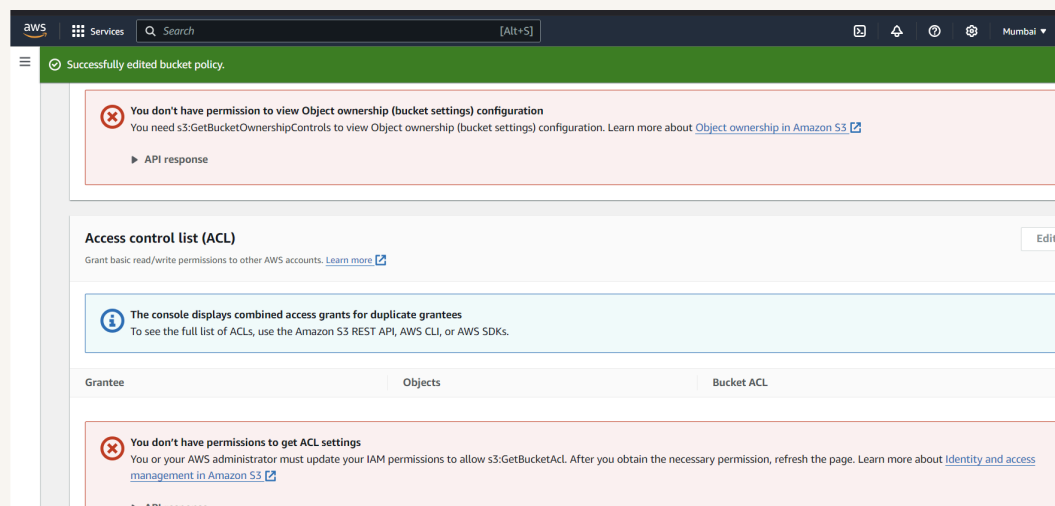




Bucket policies

Right after saving my bucket policy, my S3 bucket page showed 'denied access' warnings. This was because the policy restricts access to the bucket only from the specified VPC endpoint, blocking all other access, including from the AWS Console.

I also had to update my route table because without a route directing traffic to the VPC endpoint, the EC2 instance was trying to access S3 through the public internet. Adding the route ensured traffic would go through the secure VPC endpoint instead

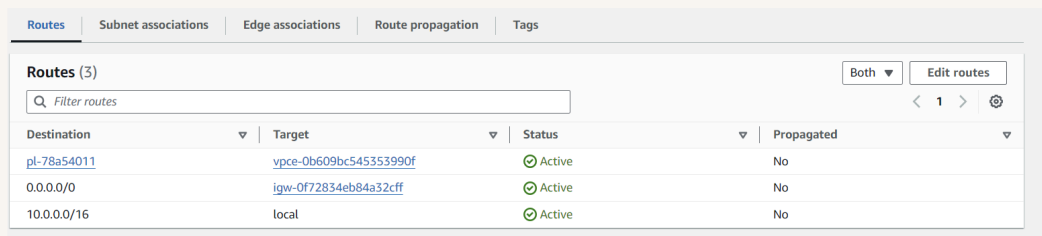




Route table updates

To update my route table, I added a route that directs traffic destined for S3 to the VPC endpoint. This ensured that any traffic from my EC2 instance trying to access S3 would securely go through the endpoint rather than the public internet.

After updating my public subnet's route table, my terminal could return the list of objects from the S3 bucket successfully. This confirmed that the VPC endpoint was properly configured and secure communication was established.

A screenshot of the AWS Management Console 'Routes' page. The page has tabs for 'Routes', 'Subnet associations', 'Edge associations', 'Route propagation', and 'Tags'. The 'Routes' tab is selected. Below the tabs, there's a section titled 'Routes (3)' with a search bar labeled 'Filter routes'. To the right of the search bar are buttons for 'Both' (with a dropdown arrow) and 'Edit routes'. Below this is a table with four columns: 'Destination', 'Target', 'Status', and 'Propagated'. The table contains three rows of route information.

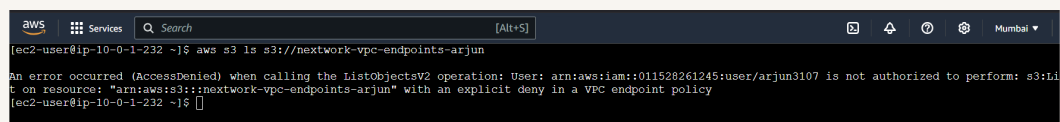
Destination	Target	Status	Propagated
pl-78a54011	vpce-0b609bc545353990f	Active	No
0.0.0.0/0	igw-0f72834eb84a32cff	Active	No
10.0.0.0/16	local	Active	No



Endpoint policies

An endpoint policy is a policy attached to a VPC endpoint that defines the actions allowed or denied when accessing AWS services. It controls traffic permissions, ensuring secure and restricted communication between your VPC and other services.

I updated my endpoint's policy by restricting access to only allow traffic from my VPC endpoint. I could see the effect of this right away because my EC2 instance was still able to access the S3 bucket, while all other traffic was blocked.



```
aws
Services
Search [Alt+S]
Mumbai
[ec2-user@ip-10-0-1-232 ~]$ aws s3 ls s3://nextwork-vpc-endpoints-arjun
An error occurred (AccessDenied) when calling the ListObjectsV2 operation: User: arn:aws:iam::011528261245:user/arjun3107 is not authorized to perform: s3:List on resource: "arn:aws:s3:::nextwork-vpc-endpoints-arjun" with an explicit deny in a VPC endpoint policy
[ec2-user@ip-10-0-1-232 ~]$
```



NextWork.org

Everyone should be in a job they love.

Check out nextwork.org for
more projects

